Lab 3, Task 3: Compute cost (hand calculations)

Fashion-MNIST input 1×28×28, batch size 1. Convention (Reddi Ch. 4): one multiply-add = 2 FLOPs, so FLOPs = 2 × MACs. Biases, ReLU, max-pool, softmax and LayerNorm are left out of the FLOP count because they are tiny next to the matrix products.

1. Parameter count (weights + biases)

MLP: 784 → 256 → 128 → 10

```
Linear 784→256 : 784·256 + 256 = 200,704 + 256 = 200,960
Linear 256→128 : 256·128 + 128 =  32,768 + 128 =  32,896
Linear 128→10  : 128·10  + 10  =   1,280 +  10 =   1,290
                                          Total = 235,146
```

CNN: Conv(1→16) → Pool → Conv(16→32) → Pool → Linear(32·7·7 → 10)

```
Conv 1→16, 3×3   : 3·3·1·16  + 16 =    144 + 16 =    160
Conv 16→32, 3×3  : 3·3·16·32 + 32 =  4,608 + 32 =  4,640
Linear 1568→10   : 1568·10   + 10 = 15,680 + 10 = 15,690
                                       Total = 20,490
```

Attention: 16 patches of 7×7, d_model = 64, 4 heads, FFN 128, 2 layers

```
Patch embedding 49→64          : 49·64 + 64           =  3,200
Learned position embedding     : 16·64                =  1,024
One encoder layer:
  Q,K,V projection (in_proj)   : 64·(3·64) + 3·64     = 12,480
  output projection            : 64·64 + 64           =  4,160
  FFN Linear 64→128            : 64·128 + 128         =  8,320
  FFN Linear 128→64            : 128·64 + 64          =  8,256
  2 × LayerNorm (γ, β)         : 2·(64 + 64)          =    256
                                     one layer        = 33,472
Two encoder layers             : 2 · 33,472           = 66,944
Classifier Linear 64→10        : 64·10 + 10           =    650
                                                Total  = 71,818
```

Model	Hand count	torchinfo.summary	Inside 100k–150k target?
MLP	235,146	235,146	No (above)
CNN	20,490	20,490	No (below)
Attention	71,818	71,818	No (below)

The hand counts match torchinfo exactly. The layer sizes fixed in the manual do not land in the 100k–150k target band. They were kept as specified.

2. FLOPs for one layer of each type

(a) Linear layer: MLP first layer, 784 → 256

```
FLOPs = 2 × in × out = 2 × 784 × 256 = 401,408  ≈ 0.40 MFLOPs
```

(b) Conv2d layer: CNN second conv, 16 → 32, 3×3, padding 1, input 14×14

```
H_out = (H_in + 2p − K)/s + 1 = (14 + 2 − 3)/1 + 1 = 14   →   H_out = W_out = 14
FLOPs = 2 × K × K × C_in × C_out × H_out × W_out
      = 2 × 3 × 3 × 16 × 32 × 14 × 14
      = 2 × 4,608 × 196 = 1,806,336  ≈ 1.81 MFLOPs
```

(c) Attention: one Transformer encoder layer, seq = 16, d_model = 64, h = 4, d_ff = 128

```
QKV projections   : 2 × seq × d_model × 3·d_model = 2 × 16 × 64 × 192 = 393,216
Scores Q·Kᵀ       : 2 × seq² × d_model           = 2 × 256 × 64      =  32,768
                    (per head 2·16·16·16 = 8,192; × 4 heads = 32,768, so heads do not change the total)
Weights × V       : 2 × seq² × d_model           = 2 × 256 × 64      =  32,768
Output projection : 2 × seq × d_model × d_model  = 2 × 16 × 64 × 64  = 131,072
FFN (64→128→64)   : 2 × seq × d_model × d_ff × 2 = 2 × 16 × 64 × 128 × 2 = 524,288
                                                   one layer total = 1,114,112 ≈ 1.11 MFLOPs
```

Most of the layer's work is in the dense projections (QKV 35%, FFN 47%). The seq^2 terms are only about 6% because the sequence is short (16 tokens).

3. Whole-model FLOPs (batch 1)

```
MLP       = 2·784·256 + 2·256·128 + 2·128·10
          = 401,408 + 65,536 + 2,560                          =   469,504

CNN       = conv1 2·3·3·1·16·28·28 + conv2 1,806,336 + fc 2·1568·10
          = 225,792 + 1,806,336 + 31,360                      = 2,063,488

Attention = embed 2·16·49·64 + 2 × encoder layer + head 2·64·10
          = 100,352 + 2 × 1,114,112 + 1,280                   = 2,329,856
```

FLOPs per parameter (how many times each weight is reused in one forward pass):

```
MLP  469,504 / 235,146 ≈ 2.0     CNN  2,063,488 / 20,490 ≈ 100.7     Attention  2,329,856 / 71,818 ≈ 32.4
```

4. Hand calculation vs thop.profile

thop returns MACs, so its value is multiplied by 2 to compare. "Default" means a normal `model.eval()` run. "Fast path off" means `torch.backends.mha.set_fastpath_enabled(False)`. `torch.utils.flop_counter.FlopCounterMode` is added as a third check.

Layer / model	Hand	thop ×2 (default)	thop ×2 (fast path off)	FlopCounter (fast path off)
Linear 784→256	401,408	401,408	401,408	401,408
Conv2d 16→32	1,806,336	1,806,336	1,806,336	1,806,336
Encoder layer	1,114,112	0	540,672	1,048,576
MLP (model)	469,504	469,504	469,504	469,504
CNN (model)	2,063,488	2,063,488	2,063,488	2,063,488
Attention (model)	2,329,856	101,632	1,182,976	2,198,784

Explaining the gap

1. **Linear and Conv2d match exactly.** thop counts in·out MACs per output position for Linear and K·K·C_in MACs per output element for Conv2d. That is the same formula used by hand, and thop also skips the bias.
2. **thop's default count for the encoder layer is 0.** `thop.profile` calls `model.eval()`. In eval mode PyTorch sends `TransformerEncoderLayer` through one fused kernel (`_transformer_encoder_layer_fwd`), so none of its sub-modules run and no thop hook fires. For the whole model thop then sees only the patch embedding (2·50,176) and the head (2·640): 101,632. It also reports just 3,850 parameters instead of 71,818.
3. **With the fast path off, thop still misses attention.** `nn.MultiheadAttention` has no thop rule. Its QKV projection uses a raw weight tensor, and its `out_proj` is a Linear subclass that thop does not recognise. Only the FFN is counted (2·262,144 = 524,288), plus 2·4,096 that thop assigns to each LayerNorm. Total: 540,672.
4. **FlopCounterMode is 65,536 short:** 1,114,112 − 1,048,576 = 65,536 = 32,768 ($Q \cdot K^T$) + 32,768 (weights × V). On CPU the attention core runs as `scaled_dot_product_attention`, which has no FLOP formula registered in torch 2.3. All the projections are counted correctly.
5. **Hand counts leave out cheap element-wise ops** (bias, ReLU, softmax, LayerNorm, pooling). These are well under 1% of the matrix-product FLOPs.

Conclusion: for MLPs and CNNs the profiler and the formulas agree. For attention, every tool undercounts, by between 6% and 96%. The hand formula is the reliable number.

5. Measured inference latency (CPU, torch 2.3.1, 4 threads, 10 warm-up runs)

Model	Hand FLOPs / sample	Batch 1, median of 100 (ms)	Batch 128, median of 20 (ms)	Batch 128, per sample (ms)
MLP	0.47 M	0.154	0.874	0.0068
CNN	2.06 M	0.781	27.499	0.2148
Attention	2.33 M	2.083	15.440	0.1206

Latency does not follow FLOPs. The Attention model has about 1.1× the CNN's FLOPs but is 2.7× slower at batch 1, because it runs many small kernels. At batch 128 it is 1.8× faster than the CNN, because its work is dense GEMMs that batch well. The CNN instead has to write, pool and re-read large activation maps (16×28×28 and 32×14×14 per image), and its small-channel convolutions run less efficiently on CPU than plain matrix multiplies.